

Cookies Notes

Learning Sources

[MDN Web Docs - Using HTTP Cookies](#)

Note

Originally created summer 2026

Cookies

-aka. browser cookie, web cookie

-Cookie - a small piece of data server sends to browser. Browser may store, create, & mod cookies, plus return cookies to server with later requests.

-Allow browser to store small amounts of data locally. Useful as HTTP is stateless.

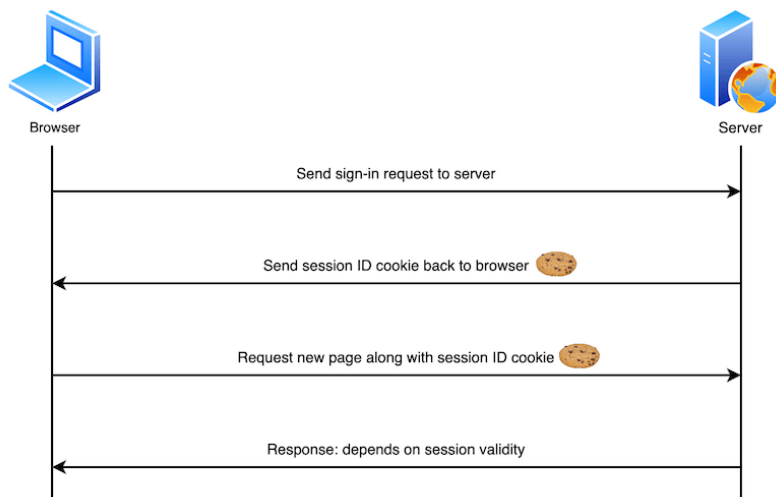
Cookie Uses

-Often used to tell if request from same browser / user or a generic request (stored auth, etc.)

-Three main uses:

- Session management - user sign-in status, shopping cart contents, etc.
- Personalization - user preferences (ex. display language, UI theme)
- Tracking - recording and analyzing user behavior

-Sign-in cookie example:



Data Storage

-While cookies in the old days were used for data storage, for modern local storage, APIs like [localStorage](#) and [sessionStorage](#) or [IndexedDB](#) are recommended instead.

-Drawbacks of cookies as data storage:

- Browsers limited to num of cookies per domain
- Max cookie size is usually 4kb

- Cookies sent w/ every request which can affect performance, especially if a lot of cookies sent

-Modern solutions are designed with storage in mind, do not need to send to server, and do not have above drawbacks

Setting & Getting Cookies

-After HTTP request, server can create and respond w/ cookie via:

Set-Cookie: <cookie-name>=<cookie-value>

-Ex. HTTP Response:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: yummy_cookie=chocolate
Set-Cookie: tasty_cookie=strawberry
```

-Can set cookie using Cookies module in Py. Similar in other langs/frameworks.

-Browser typically sends cookies back with each request:

```
GET /sample_page.html HTTP/2.0
Host: www.example.org
Cookie: yummy_cookie=chocolate; tasty_cookie=strawberry
```

Cookie Lifetime

-Can set cookie expiration time via Expires or Max-Age attribute:

```
Set-Cookie: id=a3fWa; Expires=Thu, 31 Oct 2021 07:28:00 GMT;
Set-Cookie: id=c4ghF; Max-Age=2592000
```

- Max-Age is newer and less error prone as it is based on server time versus relative to the client the cookie is being set for. Prefer it.

-Session cookie - deleted when browser session ends, but still valid as long as session is open, with no expiration set.

-Auth cookies should be set as new session cookies on each login, at minimum. This helps prevent session fixation attacks.

-To immediately expire cookie set its Expires to a past date or its Max-Age=0

-Can also clear all cookies for domain (and all of its subdomains...ex. all.bar.example.com) using:

Clear-Site-Data: "cookies"

-"Zombie attacks" can be used to re-create cookies, so beware what you store and how you store it

Updating Cookies

-Update a cookie data value server-side via: *Set-Cookie: id=new-value*

-Can update cookie client-side via JS with `document.cookie` as well as view data in cookie:

```
document.cookie = "yummy_cookie=chocolate";
console.log(document.cookie) // "yummy_cookie=chocolate"
```

- Cannot update a cookie client-side w/ HTTP Cookie header, via `fetch`, etc., as this poses security risk
- Even allowing JS to update cookies can result in security risks, so beware

Cookie Security

-Cookies are by default visible and modifiable by end user. If a bad actor steals and mods a cookie, they could do sketchy things like use a user's session ID to pretend they are the user themselves.

-Basic security:

- Block cookie access
- Control cookie destination
- Control third-party cookies

Block Cookie Access

- Done using the `Secure` and `HttpOnly` attributes. Ex.

```
Set-Cookie: id=a3fWa; Expires=Thu, 21 Oct 2021 07:28:00 GMT;
Secure; HttpOnly
```

-`Secure` - All requests sent to server via HTTPS protocol and hence encrypted, unlike HTTP. If attacker accesses user machine directly, though, could still steal cookie.

-`HttpOnly` - Disallows access of cookie by JS (`document.cookie`, but on access allowed, etc.). All cross-session cookies should be `HttpOnly` to prevent cross-site scripting (XSS) attacks.

Control Cookie Destination

-Done with `Domain` and `Path` attributes. Ex.

```
Set-Cookie: id=a3fWa; Expires=Thu, 21 Oct 2021 07:28:00 GMT; Secure;
HttpOnly; Domain=mozilla.org
```

-`Domain` - specifies which server can receive the cookie. Only that server and subdomains (ex. `example.foo.bar.org`) can then receive. If do not set `Domain`, then server domain, but not subdomains may access cookie.

-Note server domain should only set `Domain` attribute for its own domain or one of its subdomains.

-`Path` - specifies which path on server may receive the cookie (ex. `/docs/web/`). Note, this does not explicitly prevent unauthorized reading by a diff path, so do not use as a security measure.

Control Third-Party Cookies

-Cross-site request - Request where current site (domain) or scheme (`http/https`) do not match site user is currently visiting. Example, I visit *MyStore.com* with but then this calls *Facebook.tracker*. Includes clicks on hyperlinks that go off current domain.

-`SameSite` - lets server specify whether/when cookies are sent with cross-site requests (third-party cookies). Has three potential values:

- **Strict** - browser may only send cookie in requests to cookie's origin site made from cookie's origin site. Use for initial nav cookies, like auth cookies and shopping carts.
- **Lax** - browser may only send cookies in requests to cookie's origin site, even if navigating from another site. Use for display config and personalization cookies.
- **None** - cookies may be sent from both origin site and cross-site requests. Useful for sending to third-party, such as ad-tech. Note that if have `SameSite=None`, must also have `Secure` set.

Privacy & Tracking

-User privacy should be protected when building a site. Setting your cookies with attributes like `SameSite=Strict` is one way to show you care about user privacy.

-The opposite to protecting user privacy is being a shady ad company, where you could do things such as track a users history across many sites, then use the data you get from them to try and sell them stuff, or to sell to clients for big bucks (ex. Facebook).

-For this reason, blocking third-party cookies and using a browser extension like Cookie Auto-delete, which automatically deletes cookies for a domain and its sub-domains once you leave the domain.

-Global regulations for user privacy also exist, such as:

- Must notify users your site uses cookies
- Allow users to opt out of some cookies
- Allow users to use the bulk of your service w/o cookies

-Best when companies explicitly disclose how they use cookies to users